

Azure SPI: An Introduction

Why Microsoft owns the Azure half of OSDU, what was built to carry it, and the ideas that make the rest make sense

OSDU Fieldnotes
September 2026

About this document

This is the introduction to Azure SPI for engineers who already know OSDU. It explains why the Azure implementation of OSDU is now Microsoft's to own, what was built to carry that responsibility, and the small set of ideas that make the rest of the system make sense.

It is deliberately short and deliberately shallow. Two companion guides go deep. One covers the SPI Stack, the environment where the Azure implementation runs and is proved. The other covers the engineering system that keeps the Azure implementation alive. Those guides own the mechanisms: the branch model, the workflows, the Git operations, the infrastructure ordering, the identity configuration, the incidents, and the recovery procedures. This document names the things those guides explain and says why each exists. Where it mentions a mechanism at all, it stops at one sentence.

Read this first. Then read the guides with a question in mind.

1. Where you are starting from

You know OSDU from the inside. You know the services: partition, entitlements, legal, schema, storage, search, indexer, file, and workflow. You know the APIs, you know what a data partition is, and you know why entitlements sits underneath nearly everything. You may know Azure Data Manager for Energy, ADME, as the managed product Microsoft runs on top of all of this.

What you may not have seen is the engineering behind the Azure half of OSDU: where the Azure implementation comes from, where it runs while it is being developed, and how anyone knows it works before it is released. That is what Azure SPI is.

It is not a new data platform. The APIs do not change. A partition lookup for a partition named `opendes` is the same request it always was. What changes is who owns the code that answers it, and what it takes to keep that code current and proven. The rest of this document is about that change and what follows from it.

2. What changed

The provider model

OSDU was designed to run on any cloud, and that ambition shaped its source code. Every service needs somewhere to store things: a document database for records, a graph for entitlements, object storage for files, a search engine, a message bus, a secret store, an identity provider. OSDU's answer is a provider model. The business logic of each service is shared, and the cloud-specific plumbing sits behind swappable implementations.

In the source tree this is a directory convention. The partition service has a partition-core module holding the shared logic, and beside it provider directories: partition-azure, partition-aws, partition-gc, and others. One set of rules, several sets of plumbing. The boundary between them is a service provider interface, and that seam is what gives everything in this document its name.

Implementing a provider is not configuration. It is real code: the Azure implementation of every storage, messaging, and identity operation each service needs, kept compiling and passing tests against shared logic that other people change daily.

Venus: the community steps back from the clouds

For years the Azure provider lived in the community's GitLab repositories beside everyone else's, driven by the same shared pipeline, and Microsoft engineers contributed to it there. That coupling cost the community in three ways its own records name. A shared dependency could not be upgraded until every provider's SDK moved with it, so one lagging provider held vulnerabilities open for all. Every community change paid for provider build and deploy lanes it did not use. And community-governed and provider-maintained work had different owners and cadences but shared one permission model, because they shared one repository.

OSDU's answer is ADR 61, "Venus Release Structure and Repository Strategy," and the direction it set is now recorded on the OSDU Forum's program portal. The codebase splits into two lines. **Venus** is the community line: the shared core service code, a single community implementation of the provider interfaces maintained in the open, and the acceptance-test harness that proves them. It carries no cloud-provider-specific code. **Mercury** is the existing provider-oriented line, which moves to security and critical-fix maintenance for a published window. The decision was to remove cloud-provider code from the existing repositories entirely and leave the community implementation.

That community implementation is **CIMPL**, the OSDU Community Implementation: core services plus one open implementation of their interfaces, backed by ordinary in-cluster middleware rather than any cloud's managed services. Microsoft contributed the deployment tool that runs it, **cimpl-stack**, which stands up a CIMPL environment on any Kubernetes cluster, local or cloud, with no cloud-provider technology in the deployment path. It is a reference architecture in the literal sense: the community can build, prove, and release its services on an environment it operates itself.

Venus is a strategy, not a team. The work is organized into chartered subprojects: Core Services owns the shared services and their pipeline, Cimpl Infra owns the deployment tooling, a provider-extensions subproject owns the per-provider implementations, and QA owns the quality gates. For a cloud provider,

the message is direct: the community keeps the shared code and proves it on CIMPL, and the provider's implementation is the provider's responsibility from now on.

For Microsoft, three things follow from Venus, and they are the reason Azure SPI exists.

The Azure provider becomes Microsoft-owned code. After the separation, the Azure directories no longer exist upstream. What Microsoft holds is what Microsoft maintains, on its own schedule, to its own review and security standards, released as artifacts Microsoft can support. ADME depends on that code. So does anyone else running OSDU on Azure.

The community keeps moving. Compatibility with the reference implementation is the entire value of an OSDU implementation. An Azure implementation that stops tracking the community is a proprietary archive with extra steps. So the Azure code must keep receiving upstream change, continuously, while owning a subtree upstream will never touch again. A fork that is a snapshot fails here. What is needed is a fork that is a relationship.

Proving it becomes Microsoft's job. The community pipelines knew how to run the Azure tests. When the code leaves, that knowledge leaves with it. CIMPL proves the shared code against in-cluster middleware; it cannot prove the Azure provider, because the Azure provider's whole purpose is to talk to services CIMPL does not have. Proving a provider works is not a unit-test question. It needs a real Cosmos DB, a real Service Bus, real Storage, real Key Vault, real Entra ID, and the other services running beside it. Somebody has to be able to produce that environment on demand, and tear it down again.

Keep it alive; prove it works. Everything described in the rest of this document was built for one of those two obligations, plus the one place where they meet.

Two worlds, one seam

This is the picture to hold, because it describes where an engineer's work goes.

On one side is the community: GitLab, the Venus repositories, the shared core code, and CIMPL as the place that code runs and is proved. An engineer who changes shared logic, an interface, or a data model does that work upstream, in the open, and it is proved against the community implementation.

On the other side is Azure SPI: GitHub, the service forks, the Azure provider, and the SPI Stack as the place that provider runs and is proved. The shared code arrives from upstream by synchronization. The Azure provider hooks into it through the same interfaces the community implementation hooks into, and points them at Azure's managed services instead: Cosmos DB where CIMPL has PostgreSQL, Service Bus where it has RabbitMQ, Storage where it has MinIO, Entra ID where it has Keycloak.

The interface is the seam between the two worlds. Shared code crosses it in one direction only, from the community into the fork. Azure code never crosses it at all. An engineer may work on both sides in the same week, and the question of which side a change belongs on is the first question to ask.

3. Three things called SPI

The abbreviation is used for three different things, and confusing them makes everything harder to follow. They are distinguished here once.

The interface is the original meaning: OSDU's own pluggability seam, the boundary in the code between shared service logic and a cloud-specific implementation. It belongs to the community. It lives inside every service repository.

The engineering system, `osdu-spi`, is a Microsoft project that answers the question: where does the Azure code come from, and how does it stay current with a community that is still moving? It produces the service repositories and the container images.

The Stack, `osdu-spi-stack`, is a Microsoft project that answers a different question: how do I get a working OSDU on Azure to develop against and test against? It produces running environments.

In one sentence: the interface is the seam in the code, the engineering system keeps Microsoft's side of that seam alive, and the Stack is where that side is proved.

Two cautions. The interface is not a service and not a network hop. The shared logic and the Azure implementation are compiled into the same container image, and a request never leaves the process to cross it. And the three meanings do not map onto three repositories. The interface is inside each service repository; the two projects are the machinery around it.

4. The unit of work is a service fork

This is what you will open in the morning.

There is one GitHub repository per service: `osdu-spi-partition`, `osdu-spi-storage`, `osdu-spi-entitlements`, and so on, eight in all. Each is a fork of the corresponding community service. None is a copy.

Two owners in one tree

Inside a service fork, ownership is not uniform. Some directories are upstream-owned: the service's core module, the community acceptance tests, the shared build configuration. Those are regenerated from the community continuously. Other directories in the same repository, at the same commit, are fork-owned: the Azure provider, its Azure tests, the build definition, and the engineering configuration. Those were seeded once, from the last community revision that still contained them, and are Microsoft's from that moment on. Synchronization never writes to them again.

Ownership runs through the tree, not around the repository. A single commit contains code with two provenances and two maintenance contracts. Everything the engineering system does has to respect that line, on every run, forever.

How upstream change arrives

Upstream change arrives daily, as a reviewable pull request. It lands in a branch that is generated from the community tree rather than merged from it: shared code in, the Azure provider deliberately absent. Because the generated branch has never contained the Azure implementation, upstream deleting its Azure directory instructs the fork to delete nothing. The protection is structural. Nobody has to remember which files to rescue.

That protection is not compatibility. When upstream changes an interface that the Azure implementation depends on, the Azure code still has to be changed by a person. That integration work, combining new shared code with the fork-owned provider and making the result build and pass, is the central work of a fork. It happens in a workspace branch that is allowed to break, and reaches the protected main branch only after review and validation. Three long-lived branches, three different jobs. The guide explains the machinery.

One template, eight forks

The workflows that do all of this are not written per service. They come from `osdu-spi`, which is a template repository. A fork created from it inherits the complete engineering system, and when the template changes, the change arrives in every fork as a pull request. A fix made once lands everywhere. Adding a ninth service adds no new patterns.

What a fork produces

A fork produces a container image, built from the shared code and the Azure provider together, published to a registry, identified precisely. That image is the thing that leaves the engineering system. It is also, at that moment, only a candidate. Nothing about a successful build says the image works against Azure.

A week in a fork

Most weeks in a fork look like this. A synchronization pull request arrives carrying upstream's latest shared code; someone reads what changed and decides whether the fork takes it now. If upstream moved an interface, the Azure provider stops compiling against the new shared code, and fixing that is the week's real work. A provider fix of the fork's own, a bug in how the Azure implementation talks to Storage, say, is an ordinary pull request against the fork-owned subtree. Either kind of change builds an image, and the validation result says which questions that image has answered so far. When a change to the shared logic itself is needed, the fork is the wrong place: that change goes upstream, is proved on CIMPL, and comes back through synchronization. The fork's automation handles the routine; the judgment about what to take, what to fix, and where a change belongs stays with people.

5. The place to prove it is a Stack

An image is a candidate, not a conclusion. Turning it into a conclusion needs somewhere real to run it.

The Stack turns an Azure subscription into a running OSDU with one command. A Kubernetes cluster, the Azure data services, the in-cluster middleware, the OSDU services, working ingress, and enough seeded data that the APIs answer real requests. It takes most of an hour, and most of that hour is Azure building the cluster.

Why real Azure

The Stack uses real Cosmos DB, Service Bus, Storage, Key Vault, and Entra ID, and it is Azure-only by design. This is not a preference. The code under test is code that talks to those services. Testing the Azure provider against substitutes would bypass the very code the environment exists to prove. Running the Stack on another cloud would be a fork of the Stack, not a flag.

Two ways it is used

An engineer creates a personal environment, works against it, and deletes it. The fork repositories' automation uses a standing shared environment as a test target, borrowing it briefly and giving it back. A large part of the Stack's design exists to let eight forks do that without breaking each other.

What it is not

The Stack is not ADME and it is not a production configuration. Its own documentation says so and repeats it. All OSDU services share one Azure identity, so one service's Azure permissions are not isolated from another's. There is no backup or disaster recovery for the in-cluster databases. Middleware is sized for testing. It is disposable on purpose.

ADME is a product that consumes released artifacts on its own terms. The Stack is a proving ground. The engineering system's own summary of the relationship is that downstream systems such as ADME get stable release points to consume.

One distinction you will meet immediately

The command finishing and the environment being ready are different events. Provisioning Azure resources and bringing the workloads up are separate concerns with separate owners and separate timelines, and the Stack refuses to pretend otherwise. A successful command exit is not a readiness check. The guide names four owners and explains how readiness is actually stated.

6. Where the two meet

Everywhere except one place, the two projects are independent. The Stack can deploy community images with no fork involved. The engineering system synchronizes and builds with no Azure environment in sight.

The one place is this. A fork has a candidate image. It borrows a service's slot in a shared Stack environment, runs its image there temporarily, executes its acceptance tests against the live APIs, and puts back what was there before, whether the tests passed, failed, or never ran. Borrow, prove, restore.

It is worth naming now, before any detail, for one reason: it is the only place where both projects' designs have to agree. Several decisions in each project look like fussiness on their own and look like the same design from here. The environment publishes facts about itself, the fork declares what its tests need, and the engineering system joins the two at the moment of the run. The rule underneath is short: no environment values live in the repository, because the environment is rebuilt regularly and anything copied into a repository is stale by construction.

Restoration deserves one sentence of honesty. The run puts the environment back only while it still owns the slot, and a run that dies can leave work for a person. The guides say how.

7. Six ideas that keep coming back

The mechanisms are many. The ideas behind them are few, and they recur in both projects.

Upstream never stops, and you do not set its pace. The community changes daily, and the fork's job is to receive that change on a cadence rather than at a convenient moment. The tempting alternative is to synchronize when there is time, monthly or quarterly. It fails because each synchronization is then larger than the last, and the interface change that breaks the Azure provider is buried among hundreds of others. Most of the engineering system exists to make daily arrival routine.

Ownership runs through the tree. Shared code and Azure code live in one repository under different rules. Where a change belongs is decided by which subtree it touches, not by which repository it is in. The tempting alternative is two repositories, one for shared code and one for Azure code. It fails because the provider only compiles against a particular version of the shared code, and the two have to be built and tested together as one service.

Generate; do not merge. The upstream branch is computed from the community tree rather than merged from it. The tempting alternative is an ordinary merge from upstream. It fails the day upstream deletes its Azure directory: a merge would faithfully carry that deletion into the fork, and someone would have to notice and rescue the provider on every synchronization, forever. A generated tree that never contained the provider has nothing to delete.

Declared state and observed state differ. A command, a configuration write, and a running workload are three different events. Something declares what should be true; something else reconciles toward it; observation says what is true now. The tempting assumption is that a command that exited successfully has produced the thing it described. It fails because provisioning and convergence have different owners and different clocks, and the Stack refuses to hide that. The borrow is built on the same distinction.

Evidence has a scope. A build proves an image exists. A unit test proves a path in isolation. A deployment proves an image is running. An acceptance suite proves declared behavior in an assembled environment. The tempting reading is that green means done. It fails because a lane that was skipped for a good reason is also green, and a summary that passed may have answered a narrower question than the one being asked. Ask which question a check answered before deciding what it proves.

Halt on the unknown; make failure visible. Both projects refuse to proceed on an assumption. Synchronization stops on a source path it cannot classify. The deploy gate consults the live environment every run rather than trusting a stored flag. The tempting alternative is a sensible default: classify the unknown path as shared, or fall back to a community image when the fork's image cannot be found. It fails because a default that is wrong looks exactly like a default that is right. The most quoted lesson in the engineering system's own record is that a fallback which cannot be distinguished from success is not resilience but a blindfold.

8. What changes in your day

If you have worked on OSDU in the community model, the shift is mostly one of location and responsibility. The pairs below say where each thing was and where it is now.

Where the Azure code lives. It was a directory in the community tree, beside the AWS and Google implementations. It is now a fork-owned subtree in a Microsoft repository, and it will not receive upstream fixes again unless someone ports them deliberately.

Where shared code is changed. It still changes upstream, in the community's GitLab repositories, and it is proved there on CIMPL. What is new is that the change then travels: synchronization carries it into the fork, where the Azure provider has to keep working against it.

How upstream change reaches you. You merged when you got to it, and the merge happened wherever you were working. It now arrives daily as a pull request into a generated branch, conflicts surface in a workspace branch, and integration with the Azure code is the work rather than an interruption to it.

Where a fix belongs. Service behavior changes in the service fork. Engineering machinery changes in the template. Environment creation and workload configuration change in the Stack. Three homes, and the contribution rules follow from them.

Where you prove a change. The community pipeline knew how to run the tests. Now there is a Stack you can create yourself, and a shared one your fork's automation borrows. The knowledge of how to run the tests lives with the test declaration in the fork, and the environment's facts are read at the moment of the run.

What green means. A green pipeline meant done. Now each check answers a specific question, a skipped lane can be green, and a successful command exit is not readiness. Reading a result means asking what it tested.

Who is on the hook. Upstream used to own the Azure implementation's fate. Microsoft now owns it outright, along with proving it against real Azure. That is the whole reason the two projects exist.

9. What comes next

Two deep dives follow this introduction, and this document has been careful not to do their work.

The SPI Stack guide explains the environment: the four owners and their boundaries, the Azure-only bet and the three systems that stayed in the cluster, letting Azure run the cluster, infrastructure ordering and the deliberate pause after deployment, identity as two separate problems, getting in through the gateway, making an empty OSDU useful, the shared environment and its rules, and the fifty minutes from an empty subscription to a running platform.

The engineering system guide explains the fork: becoming a fork, generating the upstream branch rather than merging into it, the daily cadence and the human steps inside it, building someone else's code, trust and credentials in automation, the customer tier, and the borrow-prove-restore handshake in full, from both sides.

The training site follows one request and one fix around the whole loop. A partition lookup for `opendes` in an environment named `dev1` goes down into the running Stack and through the Azure provider; a fix to that provider goes out to its fork, through a build, and back into the environment as a candidate. The six views are the stack, its creation, the seam inside a service, the shape of the fork, a day in the fork, and the handshake. A final collection records the reasonable assumptions that turn out to be false.

Read the guides with a question. If the question is where something runs or why it is not ready, start with the Stack. If the question is where code comes from or why a change did or did not arrive, start with the engineering system. If the question is what a green result proved, start with the handshake.

Names you will hear

OSDU. The Open Subsurface Data Universe: an open standard and open-source reference implementation for subsurface and energy data, developed under the OSDU Forum.

ADR 61. The OSDU decision, “Venus Release Structure and Repository Strategy,” to split the codebase into a community line and a provider line and remove cloud-provider code from the community repositories.

Venus. The community service line and the strategy behind it: shared core code, one open implementation of the provider interfaces, community governance through chartered subprojects, and no cloud-provider code.

Mercury. The existing provider-oriented line, which moves to security and critical-fix maintenance for a published window after the split.

CIMPL. The OSDU Community Implementation: the core services with one open implementation of their interfaces, backed by in-cluster middleware.

cimpl-stack. The community's deployment tool for CIMPL, contributed by Microsoft. It stands up a CIMPL environment on any Kubernetes cluster with no cloud-provider technology in the deployment path. A development and test tool, like the SPI Stack, but for the other side of the seam.

ADME. Azure Data Manager for Energy, Microsoft's managed OSDU product. A consumer of Azure SPI's released artifacts, not part of the engineering system or the Stack.

Provider. A cloud-specific implementation of the operations a service's shared logic needs. The Azure provider is what Microsoft now owns.

The interface. The service provider interface: the code boundary between shared logic and a provider. Inside every service, never a network hop.

Service fork. A Microsoft repository for one service, holding shared code and the Azure provider under different ownership. Eight of them.

Upstream. The community's repository for a service. The source of shared code and the thing a fork stays current with.

osdu-spi. The engineering system: a template repository whose workflows every service fork inherits.

osdu-spi-stack. The Stack: a command-line tool, infrastructure definitions, and workload configuration that produce a running OSDU environment on Azure.

Image. The container built from a service fork, holding shared code and the Azure provider together. The artifact that leaves the engineering system and enters a Stack.

Shared environment. A standing Stack environment that the forks' automation borrows to prove candidate images.

Acceptance tests. The API-level suites a fork runs against a live environment. Their declaration lives in the fork; the environment's facts are read at run time.

Borrow, prove, restore. The handshake between the two projects: a fork temporarily runs its candidate in the shared environment, tests it, and puts the environment back.

Sources and scope

This introduction is drawn from the two Microsoft projects' own documentation as of 12 September 2026, osdu-spi at revision 080f0b8 and osdu-spi-stack at revision dc2c956, together with the OSDU community's record of the Venus direction: ADR 61, the Core Services subproject's wiki, and the cimpl-stack project. The Microsoft project documentation describes the consequence of Venus, that upstream intends to remove its Azure provider source, without naming the initiative; the community sources supply the name and the reasoning. No live environment state is asserted. Mechanisms are described only to the depth needed to say why they exist; the companion guides are the reference for how they work.

1. OSDU. **ADR 61: Venus Release Structure and Repository Strategy.** The split into the Venus community line and the Mercury provider line. <https://community.opengroup.org/osdu/platform/ci-cd-pipelines/-/issues/61>
2. OSDU Core Services subproject. **About Venus Core.** What Venus and Mercury are, the subprojects, and why the coupled repositories cost the community. <https://community.opengroup.org/groups/osdu/ui/ai-devops-agent/subgroup-core/-/wikis/about>
3. OSDU. **cimpl-stack.** The community deployment tool for CIMPL: portable across Kubernetes targets, in-cluster middleware, development and test posture. <https://community.opengroup.org/osdu/platform/deployment-and-operations/cimpl-stack>
4. Azure. **OSDU SPI concepts.** The fork management problem, ownership, and the relationship to downstream consumers such as ADME. <https://azure.github.io/osdu-spi/concepts/>

5. Azure. **Engineering-system overview**. Template and fork contexts, source and artifact ownership.
<https://azure.github.io/osdu-spi/architecture/overview/>
6. Azure. **ADR-038: Upstream filter transform and one-time Azure seeding**. Why the upstream branch is generated and what it excludes. <https://azure.github.io/osdu-spi/adr/038-upstream-filter-transform/>
7. Azure. **OSDU SPI Stack architecture**. Boundaries, owners, data services, and the development and test scope.
<https://github.com/Azure/osdu-spi-stack/blob/main/docs/architecture.md>
8. Azure. **Fork deployment loop**. The borrow-prove-restore contract and its limits. <https://github.com/Azure/osdu-spi-stack/blob/main/docs/design/fork-deployment.md>
9. Azure. **ADR-041: Borrow, prove, restore lane**. The engineering system's side of the handshake.
<https://azure.github.io/osdu-spi/adr/041-borrow-prove-restore-lane/>
10. Azure. **Deployment lifecycle**. Why a successful command exit is not a readiness check.
<https://github.com/Azure/osdu-spi-stack/blob/main/docs/design/deployment-lifecycle.md>
11. OSDU Forum. **The Open Group OSDU Data Platform**. The community standard and reference implementation.
<https://osduforum.org/>